Accessibility Assessment Scheme Tool (AAST)

Frontend Developer Guide

NEXT.JS · REACT 19 · TAILWIND CSS · TANSTACK QUERY

Version: 1.0 | **Release:** 2026-06-17 | **Document ID:** AAST-DOC-FRONTEND-001

1 EXECUTIVE SUMMARY

The AAST Frontend (the "Client Shell") is a modern, responsive PWA built with **Next.js 16 (App Router)** and **React 19**. It is a **pure presentation layer with zero business logic**.

Core Principle: Zero Business Logic

✅ FRONTEND DOES	❌ FRONTEND NEVER DOES
Render UI components & API responses	Calculations, formulas, or business rules
Send user inputs via REST APIs	Hardcoded thresholds, weights, or scoring
Manage client-side navigation & state	Evaluate or interpret assessment results

Why? The backend is the single source of truth. Any business rule change requires only a database row update — never a frontend redeployment.

COMPONENT	TECHNOLOGY
Framework	Next.js 16.2.6 (App Router)
UI Library	React 19.2.6
Server State	TanStack React Query 5.x
HTTP Client	Axios 1.5
Styling	Tailwind CSS 4.x
Animation	Framer Motion 12.x
Validation	Zod 3.25
PWA	Service Worker + Web Manifest

Key Features

Assessment dashboard with interactive OBS gauge, searchable criteria grid (63 criteria), building management, certification view, PWA with offline indicator, responsive mobile-first design with bottom navigation, error boundary + Axios interceptors.

---

2 PROJECT OVERVIEW

Architecture

```
Frontend (Next.js) --> Backend (Express) --> Database (MariaDB)
     ^                      ^                      ^
  "Client Shell"        "The Kernel"        "Source of truth"
  Pure presentation     All calculations    All configurable
  Zero business logic   All validation      values
```

Page Routes (14 total)

ROUTE	PURPOSE
`/`	Home — quick links to all sections
`/dashboard`	Assessment UI with OBS gauge
`/criteria`	Searchable criteria grid
`/certification`	Filterable certification results
`/reports`	Historical evaluation reports
`/buildings`	Building list with status
`/buildings/[id]`	Building detail with evaluations
`/building`	Building type management
`/assessments/new`	Create assessment form
`/assessments/[id]`	View saved assessment
`/dimensions`	Assessment dimension management
`/disability`	Disability type management
`/login`	Authentication page

3 FRONTEND OVERVIEW

Folder Structure

```
frontend/
├── package.json / next.config.js / tsconfig.json
├── tailwind.config.ts / postcss.config.js
├── app/                              # App Router pages (14 routes)
│   ├── layout.tsx                    # Root layout (Inter font, dark theme, PWA)
│   ├── page.tsx                      # Home page
│   ├── globals.css                   # Global styles + Tailwind directives
│   ├── dashboard/page.tsx            # OBS gauge dashboard
│   ├── criteria/page.tsx             # Criteria grid
│   ├── certification/page.tsx        # Certification results
│   ├── reports/page.tsx              # History view
│   ├── buildings/
│   │   ├── page.tsx                  # Building list
│   │   └── [id]/page.tsx             # Building detail
│   ├── assessments/
│   │   ├── new/page.tsx              # Create assessment
│   │   └── [id]/page.tsx             # Assessment detail
│   ├── building/page.tsx             # Building types
│   ├── dimensions/page.tsx           # Dimensions
│   ├── disability/page.tsx           # Disability types
│   └── login/page.tsx                # Auth page
├── components/
│   ├── Header.tsx                    # Top navigation
│   ├── BottomNav.tsx                 # Mobile bottom tabs (5 tabs)
│   ├── BuildingCard.tsx              # Building summary card
│   ├── ErrorBoundary.tsx             # React error boundary
│   ├── OfflineIndicator.tsx          # PWA offline status banner
│   ├── PWAProvider.tsx               # Service worker registration
│   └── layout/
│       ├── mobile-nav.tsx            # Collapsible mobile drawer
│       └── navbar.tsx                # Desktop nav bar
├── lib/
│   ├── api/
│   │   ├── client.ts                 # Axios instance + interceptors
│   │   └── endpoints.ts              # 16 typed API wrappers
│   ├── hooks/
│   │   ├── useAssessment.ts          # TanStack Query: single assessment
│   │   ├── useBuildings.ts           # TanStack Query: buildings list
│   │   └── useCriteria.ts            # TanStack Query: criteria list
│   ├── queryClient.ts                # TanStack QueryClient config
│   ├── animations.ts                 # Shared Framer Motion variants
│   └── pwa.ts                        # Service worker registration
└── public/
```

```
    ├── manifest.json                # PWA manifest
    └── sw.js                        # Service worker
```

State Management

- **Server State:** TanStack React Query 5 — fetching, caching, refetching
- **Client State:** React `useState` for UI-local state (forms, toggles, modals)
- **Persistence:** `localStorage` under key `accessibility-assessment-state`

Hooks

HOOK	QUERY KEY	PURPOSE
`useAssessment(id)`	`['assessment', id]`	Fetch single assessment
`useBuildings()`	`['buildings']`	Fetch all buildings with evaluations
`useCriteria()`	`['criteria']`	Fetch all 63 criteria

4 API CONSUMPTION

Axios Client (`client.ts`)

```
const api = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL || "http://localhost:4000/api/v1",
  timeout: 15000,
  headers: { "Content-Type": "application/json" },
});

// Response interceptor normalizes errors into { status, ok, error, message }
api.interceptors.response.use(
  (response) => response,
  (error) => Promise.reject({
    status: error.response?.status || 500,
    ok: false,
    error: error.response?.data?.error || "Network error",
    message: error.message,
  })
);
```

Endpoints (`endpoints.ts`) — 16 typed wrappers

```
export const endpoints = {
  evaluate:     (payload) => unwrap<EvaluationResult>(api.post('/evaluate', payload)),
  getCriteria:  ()        => unwrap<Criterion[]>(api.get('/criteria')),
  createCriterion: (p)    => unwrap<Criterion>(api.post('/criteria', p)),
  getBuildingTypes: ()    => unwrap<BuildingType[]>(api.get('/building-types')),
  getNebThresholds: ()    => unwrap<NebThreshold[]>(api.get('/neb-thresholds')),
  getConfig:    ()        => unwrap<ConfigItem[]>(api.get('/config')),
  getDisabilityTypes: ()  => unwrap<Label[]>(api.get('/disability-types')),
  getAssessmentDimensions:()=>unwrap<Label[]>(api.get('/assessment-dimensions')),
  getBuildings: ()        => unwrap<Building[]>(api.get('/buildings')),
  getBuilding:  (id)      => unwrap<Building>(api.get(`/buildings/${id}`)),
  createAssessment: (p)   => unwrap<Assessment>(api.post('/assessments', p)),
  getAssessment:(id)      => unwrap<Assessment>(api.get(`/assessments/${id}`)),
  getReports:   ()        => unwrap<Report[]>(api.get('/reports')),
  login:        (p)       => unwrap<AuthResult>(api.post('/auth/login', p)),
};
```

Using Endpoints

```
// Direct call (event handler)
const result = await endpoints.evaluate({ buildingType, scores });

// Via TanStack Query (recommended)
const { data, isLoading, error } = useQuery({
  queryKey: ['criteria'],
  queryFn: () => endpoints.getCriteria(),
});
```

TanStack Query Config

```
const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 5 * 60 * 1000,   // 5 min
      retry: 1,
      refetchOnWindowFocus: false,
    },
  },
});
```

---

5 REQUEST LIFECYCLE

```
User -> Next.js Page -> TanStack Query Hook -> Axios Client -> Backend Express
                                                                    |
User <- React Re-render <- Hook updates cache <- Axios unwrap <- JSON response
```

Step-by-step:

1. User interacts (selects building type, rates criteria, clicks "Evaluate")
2. Page component collects form state, calls hook mutation
3. TanStack Query hook calls `endpoints.evaluate()` via Axios
4. Axios sends POST to `NEXT_PUBLIC_API_URL` with 15s timeout, JSON headers
5. Response interceptor normalizes into `{ status, ok, error, message }`
6. Hook updates cache, triggers React re-render
7. User sees OBS gauge, NEB class, strengths, weaknesses

---

6 LOCAL DEVELOPMENT SETUP

Prerequisites

TOOL	MIN VERSION	CHECK
Node.js	18 LTS (20 rec.)	`node --version`
npm	9+	`npm --version`

> Backend must be running. See Backend Developer Guide for backend setup.

Quick Start

```
# 1. Install
git clone <repo-url> && cd aast-project/frontend && npm install

# 2. Configure (optional — defaults to localhost:4000)
echo "NEXT_PUBLIC_API_URL=http://localhost:4000/api/v1" > .env.local

# 3. Ensure backend is running
cd ../backend && docker compose up -d mariadb && npx ts-node src/scripts/seed.ts && npm
start

# 4. Start frontend
cd ../frontend && npm run dev
# Output: ▲ Next.js 16.2.6 — Local: http://localhost:3000

# 5. Verify — open http://localhost:3000, navigate to Dashboard, run an evaluation
```

Environment Variables

VARIABLE	DEFAULT
`NEXT_PUBLIC_API_URL`	`http://localhost:4000/api/v1`

NPM Scripts

SCRIPT	PURPOSE
`npm run dev`	Dev server with HMR + Fast Refresh
`npm run build`	Production build (optimized bundles)
`npm start`	Production server on port 3000
`npm run lint`	ESLint

Setup Troubleshooting

SYMPTOM	FIX
"Network error"	Start backend on port 4000
CORS errors	Check `NEXT_PUBLIC_API_URL`
Port 3000 in use	Kill other dev servers
White screen	Check browser Console (F12)
Styles broken	Check `postcss.config.js` + `tailwind.config.ts`

7 COMMON WORKFLOWS

1. Running an Assessment (User Flow)

Open Dashboard → select building type → rate criteria 1–5 → click "Evaluate" → review OBS gauge, NEB class, strengths/weaknesses → optionally save.

2. Adding a New Page

Create `app/feature/page.tsx` → add TanStack Query hook if needed → add endpoint to `endpoints.ts` if needed → add nav link in `Header.tsx` or `BottomNav.tsx` → test with `npm run dev`.

3. Adding a New API Endpoint

```
// lib/api/endpoints.ts
getNewResource: () => unwrap<NewType[]>(api.get('/new-resource')),
```

4. Creating a Reusable Component

Create `components/MyComponent.tsx` → add `"use client"` if interactive → export default → import in pages.

5. Styling with Tailwind

```
<div className="flex items-center gap-4 p-6 bg-white rounded-lg shadow-md">
  <h2 className="text-xl font-semibold text-gray-900">Title</h2>
</div>
```

Mobile-first responsive, dark mode via Tailwind classes, shared variants in `lib/animations.ts`.

6. Debugging API Calls

Open DevTools (F12) → Network tab → filter by `/api/v1` → inspect request headers, payload, response. Check Console for Axios interceptor errors.

7. PWA Testing

Use Lighthouse PWA audit. Verify `manifest.json` is valid, `sw.js` is registered, site served over HTTPS or localhost.

---

8 DEBUGGING GUIDE

Top 5 Errors

1. "Network Error" in Browser

Fix: Backend not running — start backend. Check `NEXT_PUBLIC_API_URL`. Ensure `cors()` middleware is active.

2. "Hydration Error" in Console

Fix: Server/client rendered different HTML. Use `"use client"` or `useEffect` for browser-only code (`window`, `document`).

3. "useSearchParams() should be wrapped in Suspense"

Fix: Wrap component in `<Suspense fallback={<div>Loading...</div>}>`.

4. White Screen / Blank Page

Fix: Check browser Console for JS errors, Network tab for failed API calls, verify `ErrorBoundary` is wrapping the tree.

5. TanStack Query Stale Data

Fix: Use `queryClient.invalidateQueries()` after mutations, check `staleTime` in `queryClient.ts`, force refetch with `refetch()`.

Log Locations

COMPONENT	HOW TO VIEW
Frontend Console	F12 → Console tab
Network Requests	F12 → Network tab
React Components	React DevTools extension
TanStack Query	Built-in devtools (dev only)
Build Errors	Terminal where `npm run dev` runs

9 TROUBLESHOOTING

#	PROBLEM	LIKELY FIX
1	Can't reach API	Check `NEXT_PUBLIC_API_URL`, verify backend running
2	CORS errors	Ensure backend `cors()` middleware; check origin
3	White screen	Check browser Console for render errors
4	PWA not installing	Validate `manifest.json` + `sw.js`; use Lighthouse
5	Hydration mismatch	Add `"use client"` or `useEffect` for browser-only code
6	Styles broken	Check postcss + tailwind config; restart dev server
7	Build fails	`npm ci` then `npm run build`; check Next.js version
8	Slow page load	Audit bundle size + API response time; use Lighthouse
9	Infinite re-render	Check useEffect dependencies; use React DevTools profiler
10	Form state lost	Persist to `localStorage` or query params

10 CODING STANDARDS

Core Principles

1. **Zero Business Logic:** Never implement calculations, formulas, or thresholds in the frontend
2. **TypeScript Strict:** Explicit types, no `any`, interfaces for API responses
3. **Server vs Client:** Minimize `"use client"` — prefer Server Components
4. **Component Composition:** Small, focused, single-responsibility components

Naming Conventions

ELEMENT	CONVENTION	EXAMPLE
Components	PascalCase	`BuildingCard`
Hooks	camelCase, `use` prefix	`useBuildings`
Functions/variables	camelCase	`fetchBuilding()`
Types/Interfaces	PascalCase	`EvaluationResult`

Component Rules

- Functional components only (no classes)
- `"use client"` only when using `useState`, `useEffect`, event handlers, browser APIs, or TanStack Query
- Use TanStack Query hooks for server data (never fetch directly in components)
- Use Framer Motion `motion.*` for animations
- Export default for pages, named for shared components

Prohibited Patterns

```
// ❌ PROHIBITED — business logic in frontend
const obs = (rawScore / 25) * 100;
const nebClass = obs >= 85 ? 'A+' : 'A';
const weights = [0.2, 0.2, 0.2, 0.2, 0.2];

// ✅ CORRECT — pass-through to backend
const result = await endpoints.evaluate({ buildingType, scores });
const { obs, nebClass } = result;
```

Formatting: 2-space indent, no trailing whitespace, files end with newline, semicolons required.

---

11 GIT WORKFLOW

Branch strategy: `main` (production) ← `develop` (integration) ← `feature/*` / `fix/*` / `chore/*`

Commit format: `<type>(frontend): <description>` — types: `feat`, `fix`, `chore`, `docs`, `test`, `refactor`

PR process: Create feature branch → commit → `npm run build && npm run lint` → push → PR to `develop` → CI runs `build-frontend` → code review → merge.

CI pipeline (on push to main): test-backend → build-backend → deploy-backend → build-frontend → deploy-frontend (5 jobs).

---

12 DEPLOYMENT

Azure Static Web App (Standard tier)

- **Build:** `next build` produces optimized static + dynamic output
- **Hosting:** Global CDN with automatic SSL
- **Env Vars:** Set in Azure Portal → Static Web App → Configuration

```
# Production env var
NEXT_PUBLIC_API_URL=https://app-aas-backend-production.azurewebsites.net/api/v1
```

Post-Deployment Verification

1. Open production URL, verify all pages load
2. Check Dashboard connects to production backend
3. Run test evaluation — verify OBS gauge renders
4. Test mobile responsiveness
5. Run Lighthouse audit (performance, accessibility, PWA)

---

13 FAQ

Q: Why zero business logic? A: Single source of truth in the backend. Business rule changes never require frontend redeployment.

Q: Why Next.js App Router instead of Pages Router? A: React Server Components, streaming, improved layouts — recommended for new Next.js projects.

Q: Why TanStack Query instead of Redux? A: Specializes in server state (fetch/cache/sync). Minimal client-only state means full state management is overkill.

Q: When to use `"use client"`? A: When using `useState`, `useEffect`, event handlers, browser APIs, or TanStack Query hooks.

Q: How to handle loading/error states? A: TanStack Query provides `isLoading`, `isFetching`, `isError`, `error`. Show spinners/skeletons for loading, fallback UI for errors.

Q: How to change API URL per environment? A: Set `NEXT_PUBLIC_API_URL` in `.env.local` (dev) or Azure Portal (prod).

---

14 USEFUL COMMANDS

COMMAND	PURPOSE
`npm run dev`	Start dev server (HMR)
`npm run build`	Production build
`npm start`	Production server
`npm run lint`	ESLint
`curl localhost:4000/health`	Backend health check
`curl localhost:4000/api/v1/criteria`	Get all criteria
`npx next info`	System debug info
F12	Open browser DevTools
Ctrl+Shift+E	Network tab
Ctrl+Shift+J	Console tab

15 GLOSSARY

TERM	DEFINITION
AAST	Accessibility Assessment Scheme Tool
App Router	Next.js 13+ file-based routing via `app/` directory
Axios	Promise-based HTTP client
Client Component	React component with `"use client"` — runs in browser
Server Component	React component rendered on server (default in App Router)
Framer Motion	React animation library
HMR	Hot Module Replacement — instant updates without full reload
NEB	National Evaluation Benchmark (A+, A, B, No Rating)
OBS	Overall Building Score (0–100%)
PWA	Progressive Web App — installable, offline-capable
TanStack Query	Server state management (fetching, caching, sync)
Tailwind CSS	Utility-first CSS framework
Zod	TypeScript-first schema validation
Service Worker	Background script for offline support & caching
Hydration	Attaching React event listeners to server-rendered HTML